

# Speaker Notes

Machine Learning Fundamentals · Batch 6

Naskah lisan per slide + panduan membaca rumus

Navasena AI

7 September 2026

Catatan pengajar untuk mendampingi deck `module01_slides.pdf` (88 slide). Tiap entri diberi label **Slide N/88** sesuai nomor halaman deck, berisi naskah yang bisa langsung diucapkan (**Ucapkan**) dan, kalau slide-nya memuat rumus, cara membacanya dengan lantang (**Baca rumus**). Naskah ini bukan bacaan ulang teks slide: slide memuat yang perlu terlihat, notes menyambungkan idenya dan menyebut batas berlakunya. Frame bertanda *Intuisi* enak dibawa santai dengan tempo normal; frame mekanisme (rumus, langkah algoritma, tabel metrik) dibaca lebih pelan dan sebaiknya berhenti sebentar setelah tiap butir. Sapaan yang dipakai: “kamu” saat mengajak mencoba, “kita” saat menalar bersama. Angka yang muncul di naskah ini diambil dari gambar yang benar-benar tampil di slide, jadi kalau gambarnya diregenerasi dan angkanya berubah, angka di naskah ikut disesuaikan.

## Glosarium: cara menyebut simbol dan singkatan

Pakai pembacaan ini konsisten sepanjang hari, termasuk saat menjawab pertanyaan.

| Simbol / singkatan | Dibaca (lantang)                                       |
|--------------------|--------------------------------------------------------|
| $y$                | “y” (nilai sebenarnya)                                 |
| $\hat{y}$          | “y topi” (prediksi model); $\hat{y}_i$ = “y topi ke-i” |
| $x, x_j$           | “x”, “x ke-j” (nilai feature)                          |
| $w, w_1, w_2$      | “w”, “w satu, w dua” (koefisien / kemiringan)          |
| $b$                | “b” (intercept, nilai prediksi saat x sama dengan nol) |
| $n, m$             | “n” (jumlah baris), “m” (jumlah feature)               |
| $\sum_{i=1}^n$     | “sigma, i dari satu sampai n” (jumlahkan semua baris)  |
| $\bar{x}, \bar{y}$ | “x bar”, “y bar” (rata-rata)                           |
| $\sigma$           | “sigma”; $\sigma(z)$ dibaca “sigmoid dari z”           |
| $\varepsilon$      | “epsilon” (angka kecil, mis. penstabil pembagian)      |
| $\sqrt{\cdot}$     | “akar dari ...”                                        |

| Simbol / singkatan | Dibaca (lantang)                                              |
|--------------------|---------------------------------------------------------------|
| $(\cdot)^2$        | “dikuadratkan”                                                |
| $ \cdot $          | “nilai absolut” (tanpa tanda)                                 |
| $R^2$              | “R kuadrat”                                                   |
| MAE                | dieja: “M A E” (mean absolute error)                          |
| RMSE               | dieja: “R M S E” (root mean squared error)                    |
| MSE                | dieja: “M S E” (mean squared error)                           |
| MAPE               | dieja: “M A P E” (mean absolute percentage error)             |
| AUC, ROC-AUC       | dieja: “A U C”, “R O C A U C”                                 |
| $K$ (huruf besar)  | “K besar”: jumlah tetangga di KNN, jumlah kelompok di k-means |
| $k$ (huruf kecil)  | “k kecil”: indeks urutan, mis. feature ke-k, tetangga ke-k    |
| $d(a, b)$          | “d dari a dan b” (jarak antara baris a dan baris b)           |
| $\Rightarrow$      | “maka”                                                        |

*Tip pengucapan: pecahan dibaca “pembilang, per, penyebut”; pangkat dibaca “dipangkatkan” atau “dikuadratkan”; nama metode di kode disebut apa adanya, mis. `fit()` dibaca “fit”.*

## Daftar Isi

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Pembuka</b>                                                     | <b>6</b>  |
|          | Slide 1: Judul . . . . .                                           | 6         |
|          | Slide 2: Peta hari ini . . . . .                                   | 6         |
| <b>2</b> | <b>Act 1 — Apa itu Machine Learning?</b>                           | <b>6</b>  |
|          | Slide 3: Act 1 (pembuka) . . . . .                                 | 6         |
|          | Slide 4: Intuisi: Aturan tertulis vs belajar dari contoh . . . . . | 6         |
|          | Slide 5: Tiga jenis machine learning . . . . .                     | 6         |
|          | Slide 6: Contoh yang kamu pakai tiap hari . . . . .                | 7         |
|          | Slide 7: Tools hari ini . . . . .                                  | 7         |
| <b>3</b> | <b>Act 2 — Bagaimana model belajar dan diuji?</b>                  | <b>7</b>  |
|          | Slide 8: Act 2 (pembuka) . . . . .                                 | 7         |
|          | Slide 9: Feature dan target . . . . .                              | 7         |
|          | Slide 10: Intuisi: Kenapa data dibagi dua . . . . .                | 8         |
|          | Slide 11: Data train, validation, dan test . . . . .               | 8         |
|          | Slide 12: Intuisi: Leakage . . . . .                               | 8         |
|          | Slide 13: Scaling . . . . .                                        | 8         |
|          | Slide 14: Intuisi: Apa artinya belajar . . . . .                   | 9         |
|          | Slide 15: Loss . . . . .                                           | 9         |
|          | Slide 16: Underfitting dan overfitting . . . . .                   | 9         |
|          | Slide 17: Bias–variance trade-off . . . . .                        | 10        |
|          | Slide 18: Cross-validation . . . . .                               | 10        |
|          | Slide 19: Baseline . . . . .                                       | 10        |
|          | Slide 20: Metrik regresi . . . . .                                 | 10        |
|          | Slide 21: Confusion matrix . . . . .                               | 11        |
|          | Slide 22: Precision, recall, F1, ROC-AUC . . . . .                 | 11        |
|          | Slide 23: Alur kerja . . . . .                                     | 12        |
| <b>4</b> | <b>Act 3 — Bagaimana memprediksi nilai?</b>                        | <b>12</b> |
|          | Slide 24: Act 3 (pembuka) . . . . .                                | 12        |
|          | Slide 25: Intuisi: es krim . . . . .                               | 12        |
|          | Slide 26: Linear regression (mekanisme) . . . . .                  | 12        |
|          | Slide 27: Linear regression (kapan pakai) . . . . .                | 13        |
|          | Slide 28: Regularisasi . . . . .                                   | 13        |
|          | Slide 29: Intuisi: time series . . . . .                           | 13        |
|          | Slide 30: Stasioner dan differencing . . . . .                     | 14        |
|          | Slide 31: SARIMA dan interval prediksi . . . . .                   | 14        |
|          | Slide 32: Evaluasi time series . . . . .                           | 14        |
|          | Slide 33: Ringkasan Act 3 . . . . .                                | 15        |

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>5</b> | <b>Act 4 — Bagaimana memprediksi kategori?</b>        | <b>15</b> |
|          | Slide 34: Act 4 (pembuka) . . . . .                   | 15        |
|          | Slide 35: Regresi dan klasifikasi . . . . .           | 15        |
|          | Slide 36: Intuisi: logistic regression . . . . .      | 15        |
|          | Slide 37: Threshold dan data timpang . . . . .        | 16        |
|          | Slide 38: Logistic regression (kapan pakai) . . . . . | 16        |
|          | Slide 39: Intuisi: decision tree . . . . .            | 16        |
|          | Slide 40: Decision tree (mekanisme) . . . . .         | 16        |
|          | Slide 41: Decision tree (kapan pakai) . . . . .       | 17        |
|          | Slide 42: Intuisi: KNN . . . . .                      | 17        |
|          | Slide 43: KNN (mekanisme) . . . . .                   | 17        |
|          | Slide 44: KNN (kapan pakai) . . . . .                 | 18        |
|          | Slide 45: Intuisi: SVM . . . . .                      | 18        |
|          | Slide 46: SVM (mekanisme) . . . . .                   | 18        |
|          | Slide 47: SVM (kapan pakai) . . . . .                 | 18        |
|          | Slide 48: Empat model, satu dataset . . . . .         | 19        |
|          | Slide 49: Ringkasan Act 4 . . . . .                   | 19        |
| <b>6</b> | <b>Act 5 — Bagaimana jika model bekerja sama?</b>     | <b>19</b> |
|          | Slide 50: Act 5 (pembuka) . . . . .                   | 19        |
|          | Slide 51: Intuisi: rata-rata tebakan . . . . .        | 19        |
|          | Slide 52: Bagging dan boosting (gagasan) . . . . .    | 20        |
|          | Slide 53: Intuisi: voting . . . . .                   | 20        |
|          | Slide 54: Hard voting dan soft voting . . . . .       | 20        |
|          | Slide 55: Intuisi: Random Forest . . . . .            | 21        |
|          | Slide 56: Random Forest (mekanisme) . . . . .         | 21        |
|          | Slide 57: Intuisi: boosting . . . . .                 | 21        |
|          | Slide 58: Bagging dan boosting berdampingan . . . . . | 21        |
|          | Slide 59: XGBoost . . . . .                           | 22        |
|          | Slide 60: Early stopping . . . . .                    | 22        |
|          | Slide 61: Feature importance . . . . .                | 22        |
|          | Slide 62: Stacking . . . . .                          | 22        |
|          | Slide 63: Tidak ada model terbaik . . . . .           | 23        |
|          | Slide 64: Ringkasan Act 5 . . . . .                   | 23        |
| <b>7</b> | <b>Act 6 — Bagaimana menemukan pola tanpa label?</b>  | <b>23</b> |
|          | Slide 65: Act 6 (pembuka) . . . . .                   | 23        |
|          | Slide 66: Supervised vs unsupervised . . . . .        | 23        |
|          | Slide 67: Intuisi: K-Means . . . . .                  | 24        |
|          | Slide 68: K-Means (mekanisme) . . . . .               | 24        |
|          | Slide 69: K-Means (kapan pakai) . . . . .             | 24        |
|          | Slide 70: Memilih K (inertia) . . . . .               | 25        |
|          | Slide 71: Memilih K (silhouette) . . . . .            | 25        |
|          | Slide 72: Hierarchical clustering . . . . .           | 25        |

|                                              |           |
|----------------------------------------------|-----------|
| Slide 73: DBSCAN . . . . .                   | 26        |
| Slide 74: Clustering di data nyata . . . . . | 26        |
| Slide 75: Ringkasan Act 6 . . . . .          | 26        |
| <b>8 Act 7 — Kapan GPU membantu?</b>         | <b>26</b> |
| Slide 76: Act 7 (pembuka) . . . . .          | 26        |
| Slide 77: Intuisi: CPU dan GPU . . . . .     | 27        |
| Slide 78: Yang cocok di GPU . . . . .        | 27        |
| Slide 79: cuML . . . . .                     | 27        |
| Slide 80: XGBoost di GPU . . . . .           | 27        |
| Slide 81: Kapan GPU tidak menang . . . . .   | 28        |
| Slide 82: Benchmark . . . . .                | 28        |
| Slide 83: Ekosistem NVIDIA . . . . .         | 28        |
| Slide 84: Ringkasan Act 7 . . . . .          | 29        |
| <b>9 Penutup</b>                             | <b>29</b> |
| Slide 85: Sepuluh notebook . . . . .         | 29        |
| Slide 86: Apa selanjutnya . . . . .          | 29        |
| Slide 87: Latihan mandiri . . . . .          | 29        |
| Slide 88: Terima kasih . . . . .             | 30        |

## Pembuka

### Slide 1/88 Judul: Machine Learning Fundamentals

**Ucapkan:** "Selamat pagi, dan selamat datang di hari pertama. Hari ini satu modul penuh: Machine Learning Fundamentals, sepuluh notebook, semuanya dijalankan di Google Colab. Sebelum notebook pertama dibuka, kita sepakati dulu istilah dan alur kerjanya, supaya sepuluh notebook itu tidak terasa seperti sepuluh resep yang berdiri sendiri."

### Slide 2/88 Peta hari ini

**Ucapkan:** "Ini seluruh isi hari ini dalam satu halaman. Enam kotak, sepuluh notebook. Kita mulai dari regresi di nb01, lalu empat model klasifikasi di nb02 sampai nb05: logistic regression, decision tree, KNN, dan SVM. Setelah itu dua notebook tentang menggabungkan model, nb06 dan nb07. Lalu clustering di nb08, time series di nb09, dan terakhir nb10 yang menjalankan model klasik di GPU. Tapi urutan bicaranya tidak langsung ke nb01. Satu blok pertama justru soal alur kerja: bagaimana model belajar, dan bagaimana hasilnya diuji supaya angkanya layak dilaporkan. Blok itu yang dipakai ulang di sembilan notebook sisanya."

## Act 1 — Apa itu Machine Learning?

### Slide 3/88 Act 1: Apa itu Machine Learning?

**Ucapkan:** "Blok pertama, dan yang paling singkat. Pertanyaannya: apa sebenarnya yang dikerjakan machine learning, dan kapan pendekatan ini lebih masuk akal daripada menulis aturannya sendiri."

### Slide 4/88 Intuisi: Aturan tertulis vs belajar dari contoh

**Ucapkan:** "Mulai dari contoh ini: filter spam. Versi kiri ditulis tangan. Satu aturan untuk kata GRATIS, dan besoknya kata itu ditulis dengan spasi di antara hurufnya. Satu aturan untuk pengirim tak dikenal, dan alamat barunya dibuat setiap hari. Aturannya tidak salah, hanya tidak pernah selesai. Versi kanan bekerja lain. Kita memberi sepuluh ribu email yang sudah ditandai spam atau bukan, dan selama pelatihan model menyesuaikan parameternya untuk menangkap pola pada contoh itu. Yang perlu dicatat, model tidak menebak sendiri tujuannya. Data, label, dan pilihan kita yang menentukan pola apa yang dicari. Jadi machine learning bukan program tanpa aturan, tetapi program yang aturannya dipelajari dari contoh."

### Slide 5/88 Tiga jenis machine learning

**Ucapkan:** "Tiga cabang, dan hampir seluruh hari ini ada di cabang paling kiri. Supervised berarti setiap contoh datang bersama jawabannya: email ini spam, penjualan bulan itu segini. Sembilan

dari sepuluh notebook hari ini ada di sini. Unsupervised berarti tidak ada kolom jawaban; yang dicari struktur di dalam datanya sendiri, dan itu clustering di nb08. Reinforcement learning belajar dari reward atas tindakannya, misalnya agen yang mengendalikan sesuatu berulang kali. Hari ini reinforcement hanya kita sebut sebagai konteks, tidak ada notebooknya. Yang membedakan ketiganya bukan tingkat kesulitan, tetapi bentuk data yang tersedia.”

#### Slide 6/88 Contoh yang kamu pakai tiap hari

**Ucapkan:** ”Empat layanan yang mungkin sudah kamu pakai pagi ini, dan keempatnya masuk ke kelompok model yang kita buka hari ini. Gmail memutuskan spam atau bukan, keluarannya kategori, jadi itu klasifikasi. Google Maps memperkirakan lama waktu tempuh, keluarannya angka, jadi regresi. Spotify mengelompokkan pendengar dengan selera mirip tanpa ada kolom jawaban, itu clustering. Toko online memperkirakan permintaan minggu depan dari riwayat mingguan, itu time series. Yang berbeda dari notebook kita jelas ada: ukuran datanya jauh lebih besar dan ada tim yang merawat sistemnya. Cara memilih model dan cara menilai hasilnya sama. Tanya ke kelas: dari empat baris ini, mana yang paling dekat dengan pekerjaanmu sekarang?”

#### Slide 7/88 Tools hari ini

**Ucapkan:** ”Alat yang dipakai sepanjang hari, dan daftarnya pendek. Python sebagai bahasa. Google Colab untuk menjalankan notebooknya, jadi tidak ada yang perlu dipasang di laptop; khusus nb10 runtime-nya diubah ke GPU T4. pandas untuk memuat dan melihat data, matplotlib untuk grafiknya. scikit-learn untuk hampir semua model hari ini, XGBoost untuk boosting di nb07, dan cuML untuk model klasik yang dijalankan di GPU di nb10. Satu hal yang memudahkan: semua notebook memuat datanya langsung dari URL, jadi tidak ada file yang perlu kamu unggah dulu sebelum sel pertama jalan.”

## Act 2 — Bagaimana model belajar dan diuji?

#### Slide 8/88 Act 2: Bagaimana model belajar dan diuji?

**Ucapkan:** ”Blok ini yang paling menentukan hasil kerja kita nanti, dan sering paling cepat dilewati orang. Isinya perjalanan dari kolom data mentah sampai satu angka yang layak dilaporkan ke orang lain.”

#### Slide 9/88 Feature dan target

**Ucapkan:** ”Tiga baris dari data Advertising yang dipakai di nb01. Tiga kolom pertama adalah belanja iklan di TV, radio, dan koran. Kolom terakhir, yang dipisah garis, adalah penjualannya. Tiga kolom masukan itu kita sebut feature, dan kolom yang mau diprediksi kita sebut target. Istilah ini dipakai konsisten sampai malam nanti. Lihat baris pertama: TV dua ratus tiga

puluh koma satu, radio tiga puluh tujuh koma delapan, koran enam puluh sembilan koma dua, penjualannya dua puluh dua koma satu. Karena targetnya berupa angka, tugasnya disebut regresi. Kalau targetnya kategori, misalnya nasabah merespons ya atau tidak, tugasnya klasifikasi dan targetnya kita sebut label kelas.”

#### Slide 10/88 Intuisi: Kenapa data dibagi dua

**Ucapkan:** ”Soal latihan dan soal ujian. Soal latihan dikerjakan berulang, jawabannya ada di belakang buku, dan itu memang gunanya. Soal ujian gunanya lain: nilainya baru berarti kalau soalnya belum pernah kamu lihat. Datanya kita perlakukan begitu juga. Data train dipakai untuk melatih model, boleh dilihat berulang kali. Data test disimpan utuh dan dibuka sekali saja di akhir, untuk menilai hasilnya. Satu hal yang membedakan analogi ini dari kenyataan: soal ujian dibuat orang lain, sedangkan data test kita potong sendiri dari data yang sama. Jadi disiplinnya ada di kita. Begitu data test dipakai untuk memilih model, angka dari data itu kehilangan artinya.”

#### Slide 11/88 Data train, validation, dan test

**Ucapkan:** ”Bentuk yang lebih lengkap: tiga potongan, tiga pekerjaan berbeda. Bagian hijau paling lebar, enam puluh persen, dipakai melatih model. Bagian hijau muda, dua puluh persen, dipakai membandingkan kandidat model dan memilih hyperparameter, misalnya kedalaman pohon. Bagian oranye, dua puluh persen terakhir, dibuka sekali di akhir dan angkanya itu yang dilaporkan. Yang perlu dibedakan adalah dua potongan terakhir: validation boleh dilihat berkali-kali selama kita masih memilih, test tidak. Proporsi enam puluh dua puluh dua puluh bukan aturan baku; delapan puluh dua puluh juga umum. Kalau datanya sedikit, potongan validation biasanya diganti cross-validation di data train, dan itu slide beberapa langkah lagi.”

#### Slide 12/88 Intuisi: Leakage: memakai informasi yang belum tersedia

**Ucapkan:** ”Bayangkan kamu sudah melihat kunci jawaban sebelum ujian. Nilainya tinggi, tetapi tidak lagi menunjukkan kemampuanmu menjawab soal baru. Masalah serupa muncul kalau model memakai informasi yang belum tersedia saat prediksi dibutuhkan. Di data bank ini tujuannya memprediksi respons nasabah sebelum telepon dilakukan. Kolom `duration`, lama panggilan, baru lengkap setelah panggilan berakhir. Lihat selisih kedua batang di kanan: dengan `duration`, A U C-nya nol koma sembilan tiga delapan; tanpa kolom itu, nol koma tujuh sembilan tiga. Batang kiri itu bukan model yang lebih baik, tetapi evaluasi yang memakai informasi yang tidak akan ada saat modelnya dipakai. Ini salah satu bentuk leakage. Bentuk lain, misalnya scaling yang dihitung dari seluruh data sebelum split, kita bahas di nb02.”

#### Slide 13/88 Scaling: menyamakan skala antar feature

**Ucapkan:** ”Dua panel, model yang sama, data yang sama. Yang berbeda hanya satuan sumbunya. Di kiri gaji ada di puluhan ribu dan umur di puluhan tahun, jadi saat jarak antar dua orang



dihitung, selisih gaji hampir menentukan seluruh angkanya; selisih umur sepuluh tahun tidak terasa apa-apa dibanding selisih gaji sepuluh ribu. Akurasi test di panel kiri nol koma tujuh enam dua. Setelah kedua kolom disetarakan dengan **StandardScaler**, panel kanan, akurasinya nol koma sembilan. Yang berubah bukan modelnya, tetapi arti jaraknya. Model berbasis jarak seperti KNN dan SVM, dan model dengan penalti koefisien seperti logistic regression, terpengaruh scaling. Decision tree dan random forest tidak, karena tiap pertanyaannya hanya membandingkan satu kolom dengan satu angka.”

#### Slide 14/88 Intuisi: Apa artinya “belajar”

**Ucapkan:** ”Sketsa ini yang saya harap terbawa sampai malam. Garis hijau adalah prediksi model. Titik hijau muda adalah data yang sebenarnya. Garis putus-putus oranye adalah error prediksi di tiap baris: jarak vertikal antara yang diprediksi dan yang benar-benar terjadi. Melatih model berarti menggeser garis itu, mengubah parameternya, sampai total panjang garis putus-putus itu sekecil mungkin. Jadi ketika nanti kamu memanggil `fit()`, yang terjadi di dalamnya persis ini. Satu batas yang perlu disebut: yang diperkecil adalah error di data train. Apakah garis itu juga bagus untuk data baru, jawabannya belum ada di gambar ini.”

#### Slide 15/88 Loss: satu angka yang diminimalkan

**Ucapkan:** ”Total error dari slide tadi perlu dijadikan satu angka, karena yang bisa diminimalkan hanya satu angka. Bentuk yang paling umum untuk regresi adalah Mean Squared Error. Ambil selisih antara nilai sebenarnya dan prediksi di setiap baris, kuadratkan, lalu rata-ratakan. Kuadratnya ada dua akibat. Pertama, tanda hilang, jadi kelebihan dan kekurangan tidak saling menghapus. Kedua, satu error besar menyumbang jauh lebih banyak daripada beberapa error kecil, jadi model cenderung menghindari kesalahan besar. Pelatihan mencari parameter dengan M S E terkecil di data train. Angka yang kita laporkan tetap dihitung dari data yang tidak dipakai melatih.”

**Baca rumus:**

- $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$ : “M S E sama dengan satu per n, kali, sigma, i dari satu sampai n, y ke-i kurang y topi ke-i, dikuadratkan”.
- Kalau ada yang bertanya artinya per bagian: “ $y_i$ ” dibaca “y ke-i”, nilai sebenarnya di baris ke-i; “ $\hat{y}_i$ ” dibaca “y topi ke-i”, prediksi model untuk baris itu; “n” jumlah baris.

#### Slide 16/88 Underfitting dan overfitting

**Ucapkan:** ”Lihat tiga gambar ini. Yang kiri terlalu sederhana untuk pola yang jelas melengkung: R M S E di data train nol koma empat sembilan dan di data test nol koma lima dua, dua-duanya besar. Yang kanan mengikuti hampir setiap titik, termasuk variasi acaknya: error train turun ke nol koma satu tujuh, tetapi error test naik ke nol koma dua enam. Yang tengah masih mengikuti pola utama tanpa harus melewati semua titik, dan train dengan test-nya berdekatan, nol koma dua dua keduanya. Itu gambaran awal, bukan bukti. Nanti saat kedalaman pohon kita ubah di

nb03, bandingkan hasil di data train dan data validation. Kalau perbaikannya hanya terlihat di data train, model mulai overfitting.”

#### Slide 17/88 Bias–variance trade-off

**Ucapkan:** ”Dua kurva, satu sumbu kompleksitas. Kurva hijau adalah error di data train: hampir selalu turun saat model dibuat lebih rumit. Kurva oranye adalah error di data validation: turun sampai satu titik, lalu naik lagi. Istilahnya begini. Bias berkaitan dengan penyimpangan sistematis prediksi model dari pola yang ingin dipelajari; model yang terlalu sederhana melewati pola. Variance berkaitan dengan seberapa banyak prediksi berubah kalau modelnya dilatih pada sampel yang berbeda; model yang terlalu peka terhadap data train ikut berubah banyak. Hati-hati satu hal: bias bukan sama dengan error train, dan variance bukan sama dengan selisih skor train dan test. Dua angka itu membantu mendiagnosis, tetapi bukan pengukuran langsung keduanya.”

#### Slide 18/88 Cross-validation: menilai model lima kali

**Ucapkan:** ”Kalau datanya tidak banyak, memotong satu bagian validation terasa mahal, dan hasilnya bergantung pada potongan mana yang kebetulan terambil. Cross-validation menyelesaikan keduanya. Data train dibagi lima bagian sama besar. Di putaran pertama, bagian oranye paling kiri dipakai menilai dan empat sisanya melatih. Putaran kedua, bagian oranye bergeser satu langkah. Begitu sampai lima putaran, sehingga setiap bagian pernah jadi penilai tepat satu kali. Skor akhirnya rata-rata lima putaran, jadi tidak bergantung pada satu potongan yang kebetulan mudah. Yang penting: seluruh gambar ini terjadi di dalam data train. Data test tetap di luar, belum disentuh.”

#### Slide 19/88 Baseline: pembandingan paling sederhana

**Ucapkan:** ”Sebelum melatih model apa pun, hitung dulu hasil dari cara yang paling malas. Untuk regresi: prediksi satu nilai konstan untuk semua baris, yaitu rata-rata target di data train. Untuk klasifikasi: prediksi kelas mayoritas untuk semua baris. Angka itu yang jadi garis dasar. Contohnya di `income_evaluation.csv` yang dipakai nb07: tujuh puluh enam persen barisnya berlabel kurang dari atau sama dengan lima puluh ribu. Artinya program tiga baris yang selalu menjawab label itu sudah dapat akurasi tujuh puluh enam persen, tanpa belajar apa pun. Jadi kalau modelmu dapat delapan puluh persen, yang perlu dibicarakan adalah selisih empat poin itu, bukan angka delapan puluhnya. Tanya ke kelas: berapa baseline untuk data bank di nb02?”

#### Slide 20/88 Metrik regresi: MAE, RMSE, $R^2$

**Ucapkan:** ”Tiga metrik untuk regresi, dan ketiganya menjawab pertanyaan yang sedikit berbeda. M A E adalah rata-rata selisih absolut antara prediksi dan nilai sebenarnya. Satuannya sama dengan target, jadi paling mudah diceritakan: rata-rata kita salah sekian unit penjualan. R M S E mengkuadratkan error dulu, merata-ratakan, lalu mengakarkannya, sehingga lebih peka terhadap

error besar daripada M A E. Kalau salah sedikit di banyak baris masih bisa diterima tetapi salah besar sekali tidak, R M S E yang lebih cocok. R kuadrat membandingkan error kuadrat model dengan pembanding berupa prediksi konstan rata-rata target: satu berarti prediksinya tepat pada data itu, nol berarti setara pembanding tadi, dan negatif berarti lebih buruk daripada sekadar menebak rata-rata. Pilih metriknya sebelum melatih, bukan setelah melihat hasil.”

**Baca rumus:**

- $MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$ : “M A E sama dengan satu per n, kali, sigma, nilai absolut dari y ke-i kurang y topi ke-i”.
- $RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$ : “R M S E sama dengan akar dari, satu per n, sigma, y ke-i kurang y topi ke-i, dikuadratkan”.
- $R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$ : “R kuadrat sama dengan satu kurang, sigma y ke-i kurang y topi ke-i dikuadratkan, per, sigma y ke-i kurang y bar dikuadratkan”. Sebut “y bar” sebagai rata-rata target pada data yang sedang dinilai.

#### Slide 21/88 Confusion matrix: dua jenis kesalahan

**Ucapkan:** “Sebelum membaca kotaknya, satu kesepakatan dulu: positif berarti kondisi yang ingin dideteksi, bukan hasil yang baik. Pada uji penyakit, positif berarti sakit. Pada data bank, positif berarti nasabah merespons. Dua kotak hijau adalah prediksi yang benar. Dua kotak sisanya adalah dua jenis kesalahan yang sama sekali tidak setara. False positive, kotak oranye, berarti kita bilang ya padahal tidak: alarm palsu, biayanya satu telepon yang sia-sia. False negative, kotak merah, berarti kita bilang tidak padahal ya: kasus yang terlewat, dan pada uji penyakit itu pasien yang pulang tanpa penanganan. Akurasi membagi jumlah dua kotak hijau dengan seluruh baris, dan dengan begitu menganggap dua kesalahan itu sama harganya. Karena biayanya berbeda, di banyak kasus yang kita pakai bukan akurasi.”

#### Slide 22/88 Precision, recall, F1, dan ROC-AUC

**Ucapkan:** “Empat angka, dan semuanya dibaca dari empat kotak tadi. Precision: dari semua yang kita prediksi positif, berapa bagian yang memang positif. Recall: dari semua yang benar-benar positif, berapa bagian yang berhasil tertangkap. F1 adalah rata-rata harmonik precision dan recall, dan sifatnya turun tajam kalau salah satunya rendah, jadi tidak bisa diakali dengan mengorbankan satu sisi. R O C A U C adalah peluang model memberi skor lebih tinggi pada satu contoh positif dibanding satu contoh negatif yang diambil acak. Angka di kanan berasal dari logistic regression pada data bank tanpa **duration**: lima ratus delapan puluh satu kasus positif tertangkap, tiga ratus empat puluh tujuh terlewat, seribu tujuh puluh delapan alarm palsu, dan A U C-nya nol koma tujuh sembilan tiga.”

**Baca rumus:**

- $\text{precision} = \frac{TP}{TP+FP}$ : “precision sama dengan T P, per, T P tambah F P”; sebut lantang: “true positive dibagi semua yang diprediksi positif”.

- $\text{recall} = \frac{TP}{TP+FN}$ : “recall sama dengan T P, per, T P tambah F N”; sebut lantang: “true positive dibagi semua yang sebenarnya positif”.
- $F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$ : “F satu sama dengan dua kali, precision kali recall, per, precision tambah recall”.

#### Slide 23/88 Alur kerja yang dipakai di sepuluh notebook

**Ucapkan:** “Tujuh langkah ini yang berulang di sepuluh notebook, jadi pola ini yang saya minta kamu kenali, bukan nama modelnya. Tentukan masalah dan metrik yang akan dilaporkan. Siapkan datanya: nilai kosong, outlier, dan kolom yang belum tersedia saat prediksi, seperti **duration** tadi. Split train, validation, dan test. Latih di data train. Bandingkan dengan baseline. Tuning hyperparameter dengan cross-validation di data train. Terakhir, uji sekali di data test dan laporkan angka itu. Dari pengalaman, langkah dua dan tiga memakan waktu paling banyak, dan langkah tujuh yang paling sering dilewati karena orang sudah puas dengan angka validation.”

### Act 3 — Bagaimana memprediksi nilai?

#### Slide 24/88 Act 3: Bagaimana memprediksi nilai?

**Ucapkan:** “Sekarang modelnya. Blok ini soal target berupa angka: harga, penjualan, jumlah penumpang. Dari satu garis lurus sampai deret waktu yang punya pola musiman.”

#### Slide 25/88 Intuisi: Suhu naik, es krim terjual lebih banyak

**Ucapkan:** “Tiap titik hijau adalah satu hari: suhu hari itu di sumbu bawah, jumlah es krim yang terjual di sumbu kiri. Datanya bergerak dari sekitar lima belas derajat sampai tiga puluh lima, dan penjualannya dari sekitar empat puluh sampai seratus delapan puluh buah. Polanya jelas: hari yang lebih panas cenderung menjual lebih banyak. Dua catatan sebelum lanjut. Pertama, ini pola bersama, bukan bukti bahwa suhu yang menyebabkan penjualan; hari panas juga hari libur, hari orang keluar rumah. Kedua, banyak garis bisa ditarik di antara titik-titik itu, dan yang dipilih model hanya satu. Yang menentukan pilihan itu ada di slide berikutnya.”

#### Slide 26/88 Linear regression: satu garis, dua parameter

**Ucapkan:** “Seluruh modelnya ada di satu baris. Prediksi sama dengan  $w$  kali  $x$  tambah  $b$ . Dua parameter saja untuk satu feature.  $w$  adalah kemiringan atau slope: berapa banyak prediksi berubah setiap satu satuan kenaikan  $x$ . Di contoh es krim,  $w$  kira-kira berapa buah tambahan per satu derajat.  $b$  adalah intercept, nilai prediksi saat  $x$  sama dengan nol. Kalau feature-nya lebih dari satu, tiap feature punya satu  $w$  sendiri dan bentuknya tetap penjumlahan. Pelatihan memilih  $w$  dan  $b$  yang membuat M S E di data train terkecil. Jadi ketika nb01 mencetak koefisien, yang kamu lihat adalah  $w$  untuk TV, radio, dan koran.”

**Baca rumus:**

- $\hat{y} = w x + b$ : “y topi sama dengan w kali x tambah b”.
- $\hat{y} = w_1 x_1 + w_2 x_2 + \dots + w_k x_k + b$ : “y topi sama dengan w satu x satu, tambah w dua x dua, dan seterusnya sampai w ke-k x ke-k, tambah b”.

### Slide 27/88 Linear regression: kapan pakai dan jebakannya

**Ucapkan:** “Kolom kiri: hubungan feature dan target mendekati lurus, koefisiennya perlu bisa dibaca dan dijelaskan ke orang lain, dan jumlah feature-nya masih sedikit. Kolom kanan: polanya jelas melengkung atau berjenjang, ada outlier yang menarik garisnya, atau pengaruh satu feature bergantung pada feature lain. Dua jebakan yang akan muncul di nb01. Pertama, koefisien besar tidak berarti feature itu penyebabnya; itu ukuran pengaruh menurut model pada data ini. Kedua, x sama dengan nol sering berada di luar rentang data, jadi intercept belum tentu punya arti nyata. Belanja iklan TV nol rupiah tidak pernah ada di data Advertising.”

### Slide 28/88 Regularisasi: membatasi ukuran koefisien

**Ucapkan:** “Kalau feature-nya banyak, model linear bisa mengejar error kecil di data train dengan koefisien yang besar dan saling menutupi. Regularisasi menahan itu. Yang diminimalkan bukan loss saja, tetapi loss di data train ditambah penalti atas ukuran koefisien. Ridge memakai kuadrat koefisien, jadi semua koefisien mengecil tetapi jarang sampai nol. Lasso memakai nilai absolut, dan itu bisa menolak koefisien yang tidak berguna, jadi sekaligus menyeleksi feature. Di nb01 kita lihat kolom koran yang paling dulu ditolkan Lasso. Kekuatan penaltinya satu hyperparameter, dipilih dengan cross-validation. Dan regularisasi bukan jaminan perbaikan: penalti yang terlalu kuat membuat model melewati pola penting.”

**Baca rumus:**

- Bentuk di slide: “yang diminimalkan sama dengan loss di data train, tambah penalti atas ukuran koefisien”.
- Ridge:  $\text{MSE} + \alpha \sum_j w_j^2$  dibaca “M S E tambah alfa kali sigma w ke-j dikuadratkan”.
- Lasso:  $\text{MSE} + \alpha \sum_j |w_j|$  dibaca “M S E tambah alfa kali sigma nilai absolut w ke-j”. Sebut “alfa” sebagai kekuatan penalti.

### Slide 29/88 Intuisi: Time series: urutan waktunya ikut menentukan

**Ucapkan:** “Data penumpang bulanan dari SeaPlaneTravel, dari Januari dua ribu tiga sampai Desember dua ribu empat belas. Panel atas: garis tipis data bulanan, garis tebal rata-rata bergerak dua belas bulan. Ada tiga hal yang bercampur di satu garis. Trend, arah jangka panjangnya, di sini naik terus. Pola musiman, naik-turun yang berulang tiap dua belas bulan; panel bawah merata-ratakan per bulan, dan bulan-bulan pertengahan tahun terlihat paling tinggi. Sisanya noise, gerakan yang tidak dijelaskan keduanya. Yang membuat data seperti ini berbeda dari data tabular biasa: urutan waktunya bagian dari datanya. Jadi barisnya tidak boleh diacak, dan itu

mengubah cara kita membagi data train dan test.”

### Slide 30/88 Stasioner dan differencing

**Ucapkan:** ”Dua sketsa. Di kiri deret aslinya: naik-turun musimannya ada, tetapi rata-ratanya bergeser naik terus, seperti garis putus-putus oranye itu. Di kanan, yang digambar bukan nilainya lagi, melainkan selisih antar bulan: bulan ini kurang bulan lalu. Naik-turunnya masih ada, tetapi rata-ratanya sudah tetap. Istilahnya: deret disebut stasioner kalau sifat statistiknya tetap terhadap pergeseran waktu, artinya rata-rata dan varians tidak bergantung pada kapan kita mulai mengamati. Nilai datanya sendiri tetap boleh berubah. Differencing adalah cara mendapatkannya, dan itu huruf I di SARIMA nanti. Satu peringatan: grafik yang tampak datar belum membuktikan stasioneritas. Di nb09 kita pakai uji dan lihat p-value-nya.”

### Slide 31/88 SARIMA dan interval prediksi

**Ucapkan:** ”SARIMA namanya panjang, tetapi tiap hurufnya satu gagasan yang sudah kita bahas. A R memakai nilai bulan-bulan sebelumnya sebagai masukan. I adalah differencing dari slide tadi. M A memakai error prediksi sebelumnya, jadi model belajar dari kesalahannya sendiri. S mengulang ketiganya pada jarak dua belas bulan, untuk menangkap pola musiman. Di grafik kanan, garis penuh data historis, garis putus-putus prediksinya, dan pita hijaunya interval prediksi sembilan puluh lima persen: rentang untuk nilai bulan berikutnya yang akan diamati. Perhatikan pitanya melebar makin jauh ke depan, dan itu wajar. Cakupan sembilan puluh lima persen berlaku sejauh asumsi modelnya cocok dengan datanya, jadi bukan janji bahwa setiap nilai pasti masuk rentang.”

### Slide 32/88 Evaluasi time series: split kronologis

**Ucapkan:** ”Split-nya berubah bentuk. Data train adalah bulan-bulan awal, data test dua belas bulan terakhir. Potongannya mengikuti waktu, tidak diacak, karena model tidak boleh melihat bulan yang lebih baru daripada bulan yang diprediksi. Kalau barisnya diacak seperti data tabular biasa, modelnya akan mendapat bocoran dari masa depan. Baseline-nya juga khas: seasonal naive, prediksi bulan ini sama dengan nilai bulan yang sama tahun lalu. Baseline itu sudah menangkap trend dan musiman sekaligus, jadi SARIMA baru menarik kalau mengalahkannya. Metriknya M A P E, rata-rata error relatif dalam persen, enak dibaca karena tanpa satuan. Tapi angkanya menyesatkan kalau nilai sebenarnya dekat nol, karena pembagiannya kecil.”

**Baca rumus:**

- $MAPE = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$ : “M A P E sama dengan seratus persen per n, kali, sigma, nilai absolut dari, y ke-i kurang y topi ke-i, per y ke-i”.
- Sebut lantang artinya: “rata-rata dari besarnya error dibagi nilai sebenarnya, dinyatakan dalam persen”.

### Slide 33/88 Ringkasan Act 3

**Ucapkan:** "Empat hal yang dibawa dari blok ini. Regresi memprediksi nilai berupa angka, dan linear regression mencari  $w$  dan  $b$  yang membuat total error prediksi terkecil. Koefisien memberi arah dan besar pengaruh menurut model, bukan bukti sebab-akibat. Regularisasi menambah penalti atas ukuran koefisien, dan Lasso bisa menolkan koefisien yang tidak berguna. Pada time series, urutan waktu ikut menentukan: split kronologis, differencing untuk menstabilkan rata-rata, dan seasonal naive sebagai pembanding. Yang praktis: nb01 memakai data Advertising, nb09 memakai SeaPlaneTravel."

## Act 4 — Bagaimana memprediksi kategori?

### Slide 34/88 Act 4: Bagaimana memprediksi kategori?

**Ucapkan:** "Blok terpanjang hari ini, dan empat notebook ada di dalamnya. Targetnya sekarang kategori, dan empat model yang kita pakai menarik batas keputusannya dengan cara yang berbeda-beda."

### Slide 35/88 Regresi dan klasifikasi

**Ucapkan:** "Dua sketsa, dan bedanya ada di bentuk keluaran. Di kiri, regresi: satu feature di sumbu bawah, target di sumbu kiri, dan yang dicari garis yang mengikuti titiknya. Keluarannya satu angka, seperti lama waktu tempuh atau penjualan. Di kanan, klasifikasi: dua feature jadi dua sumbu, titik merah satu kelas dan titik hijau kelas lain. Yang dicari bukan garis yang lewat di antara titik, melainkan batas yang memisahkan dua wilayah prediksi. Garis putus-putus itu batas keputusan atau decision boundary: satu titik yang melintasinya berubah prediksi kelasnya. Alur kerjanya sama seperti Act 2 tadi. Yang berubah bentuk keluaran model dan metrik yang dipakai menilainya."

### Slide 36/88 Intuisi: Logistic regression: skor jadi probabilitas

**Ucapkan:** "Bentuk S ini inti logistic regression. Sumbu bawah adalah skor yang dihitung model dari semua feature, mirip  $w$  kali  $x$  tambah  $b$  tadi, dan nilainya bisa dari minus besar sampai plus besar. Sumbu kiri adalah probabilitas, dan itu harus di antara nol dan satu. Fungsi sigmoid yang memampatkan skor tadi ke rentang itu. Skor nol jatuh tepat di nol koma lima, titik oranye di tengah. Skor besar positif mendekati satu, skor besar negatif mendekati nol. Perhatikan bentuk kurvanya: di tengah kemiringannya tajam, jadi perubahan kecil pada skor mengubah probabilitas banyak; di ujung kanan dan kiri kurvanya rata, jadi model sudah yakin dan tambahan skor tidak banyak mengubah apa-apa."

**Baca rumus:**

- Rumusnya tidak ditampilkan di slide. Kalau ada yang bertanya:  $\sigma(z) = \frac{1}{1 + e^{-z}}$  dibaca “sigmoid dari z sama dengan satu, per, satu tambah e pangkat minus z”.
- “z” di situ adalah skor model, yaitu “w satu x satu tambah w dua x dua dan seterusnya, tambah b”.

#### Slide 37/88    **Logistic regression: threshold dan data timpang**

**Ucapkan:** “Keluaran model itu probabilitas, bukan label. Label baru muncul setelah probabilitasnya dibandingkan dengan sebuah threshold, dan `predict()` di scikit-learn memakai nol koma lima. Angka itu bawaan, bukan angka baku. Kalau threshold diturunkan, lebih banyak baris diprediksi positif, jadi recall naik dan precision biasanya turun. Kalau dinaikkan, efeknya berbalik. Jadi threshold adalah pilihan yang mengikuti biaya kesalahan yang sudah kita sepakati, dan dipilih dari data validation, bukan dari data test. Satu lagi, kalau satu kelas jauh lebih banyak, `class_weight='balanced'` memperbesar bobot kelas minoritas saat melatih, sehingga kesalahan pada kelas kecil dihitung lebih mahal. Di nb02 kita coba keduanya dan lihat pengaruhnya ke precision dan recall.”

#### Slide 38/88    **Logistic regression: kapan pakai dan jebakannya**

**Ucapkan:** “Cocok kalau kamu butuh probabilitas dan bukan hanya label, kalau pengaruh tiap feature perlu bisa dijelaskan, dan kalau batas antar kelasnya mendekati garis lurus. Kurang cocok kalau batasnya jelas melengkung, kalau banyak interaksi antar feature yang tidak kamu buat sendiri sebagai kolom baru, atau kalau skala feature dibiarkan berbeda-beda tanpa scaling. Jebakannya yang paling sering kena: data bank kita sekitar delapan puluh sembilan persen berlabel tidak. Model yang selalu menjawab tidak sudah dapat akurasi delapan puluh sembilan persen dan nol recall. Jadi angka akurasi di data seperti itu tidak menjawab apa pun. Lihat precision, recall, dan A U C dulu sebelum menyimpulkan.”

#### Slide 39/88    **Intuisi: Decision tree: rangkaian pertanyaan ya atau tidak**

**Ucapkan:** “Pohon ini bisa dibaca dari atas seperti daftar pertanyaan. Pertama, umurnya lebih dari tiga puluh? Kalau ya, ke kanan, dan pertanyaan berikutnya gajinya lebih dari delapan puluh juta. Kalau tidak, ke kiri, dan ambangnya lebih rendah, lima puluh juta. Ujung jalurnya berisi prediksi: beli atau tidak beli. Yang penting dicatat, satu baris data menuruni pohon lewat satu jalur saja, dan memakai label di ujung jalur itu sebagai prediksinya. Jadi tidak ada perhitungan yang rumit saat prediksi, hanya beberapa perbandingan. Inilah model yang paling mudah ditunjukkan ke orang non-teknis: strukturnya bisa dibacakan sebagai aturan.”

#### Slide 40/88    **Decision tree: bagaimana pertanyaannya dipilih**

**Ucapkan:** “Pertanyaan di tiap simpul tidak ditulis manusia. Model mencoba banyak pasangan feature dan ambang, lalu memilih pasangan yang paling memisahkan kelas. Ukuran pemisahannya



bisa dipilih, dan default di scikit-learn adalah gini: makin kecil, makin murni isi tiap cabang, artinya satu cabang makin didominasi satu kelas. Proses itu diulang di setiap cabang sampai simpulnya murni atau sampai batas yang kamu pasang tercapai. Dua batas yang paling sering dipakai: `max_depth` untuk kedalaman, dan `min_samples_leaf` untuk isi minimum tiap ujung. Satu akibat dari cara kerja ini: karena tiap pertanyaan hanya membandingkan satu feature dengan satu angka, batas keputusannya selalu sejajar sumbu feature. Itu yang nanti terlihat di panel decision tree pada gambar perbandingan.”

#### Slide 41/88 Decision tree: kapan pakai dan jebakannya

**Ucapkan:** ”Cocok kalau aturan hasilnya perlu dibaca orang lain, kalau feature-nya bercampur angka dan kategori, dan kalau kamu tidak ingin repot dengan scaling. Pohon tidak membutuhkan scaling, dan justru lebih baik tanpa itu: di nb03 kita biarkan umur dan gaji dalam satuan aslinya supaya `plot_tree` menampilkan ambang yang bisa dibaca, misalnya umur tiga puluh dua, bukan minus nol koma empat. Kurang cocok kalau batas antar kelas miring atau melengkung halus, kalau datanya sedikit dan pohonnya dibiarkan tumbuh bebas, atau kalau kamu butuh prediksi yang stabil terhadap perubahan kecil data. Jebakannya: tanpa batas kedalaman, pohon dapat menghafal data train sampai errornya nol lalu jatuh di data baru. Pilih kedalamannya dengan cross-validation.”

#### Slide 42/88 Intuisi: KNN: lihat data terdekat, ambil suara terbanyak

**Ucapkan:** ”Titik putih dengan tanda tanya itu data baru yang belum punya label. KNN tidak punya rumus untuk itu. Yang dilakukan: lihat sekelilingnya. Lingkaran oranye memuat lima data train terdekat, tiga hijau dan dua merah, jadi prediksinya hijau. Satu hal yang perlu dibedakan dari pepatah tentang tetangga: tetangga di sini berarti data train yang paling dekat menurut jarak pada feature, bukan tetangga dalam arti sehari-hari. Kalau dua sumbunya umur dan gaji, dekat berarti umur dan gajinya mirip. Karena keputusannya berdasarkan jarak, arti jaraknya harus masuk akal dulu, dan itu kembali ke slide scaling tadi.”

#### Slide 43/88 KNN: jarak, K, dan suara mayoritas

**Ucapkan:** ”Jaraknya biasanya Euclidean, yaitu jarak lurus yang kita kenal: selisih di tiap feature dikuadratkan, dijumlahkan, lalu diakar. Prosedurnya tiga langkah: hitung jarak data baru ke data train, ambil K yang terdekat, pakai kelas terbanyak di antara mereka. K itu pilihan kita. K kecil, misalnya satu, membuat prediksi mengikuti tiap variasi acak di sekitarnya. K besar membuat batasnya terlalu rata sehingga pola lokal hilang. Di nb04 kita sapu K sampai seratus dengan cross-validation dan lihat kurvanya. Satu sifat yang unik: tidak ada parameter yang dilatih di sini. `fit()` hampir hanya menyimpan data, atau membangun struktur pencarian tetangga. Hampir semua pekerjaan terjadi saat prediksi.”

**Baca rumus:**

- $d(a, b) = \sqrt{\sum_{j=1}^m (a_j - b_j)^2}$ : “d dari a dan b sama dengan akar dari, sigma, j dari satu sampai m, a ke-j kurang b ke-j, dikuadratkan”.
- Sebut artinya: “a dan b dua baris data, m jumlah feature; selisih di tiap feature dikuadratkan, dijumlahkan, lalu diakarkan”.

#### Slide 44/88 KNN: kapan layak dicoba

**Ucapkan:** “Mulai dari data kecil dengan sedikit feature yang relevan, karena di situ KNN sering langsung memberi hasil yang wajar tanpa banyak penyetelan. Yang perlu diperhitungkan: prediksinya dapat melambat saat data train membesar, sebab pencarian tetangga dilakukan setiap kali memprediksi. Pada pencarian brute-force, setiap data baru dibandingkan dengan seluruh data train; ada struktur pencarian lain yang lebih cepat, tetapi bebannya tetap ada di waktu prediksi, bukan waktu latih. Dan karena KNN mengukur jarak, kalau skala feature berbeda, lakukan scaling agar perhitungan jaraknya tidak didominasi satu feature. Gaji dalam juta rupiah dan umur dalam tahun: tanpa scaling, selisih gaji hampir menentukan seluruh jaraknya.”

#### Slide 45/88 Intuisi: SVM: cari jalan paling lebar di antara dua kelas

**Ucapkan:** “Kalau dua kelompok titik ini bisa dipisahkan banyak garis, garis mana yang paling masuk akal? SVM menjawab: yang jaraknya ke kelas terdekat di kedua sisi paling lebar. Bayangkan bukan menarik garis, tetapi membangun jalan selebar mungkin di antara dua kelas, lalu garis batasnya ada di tengah jalan itu. Titik yang menempel di tepi jalan disebut support vector, dan di gambar itu yang diberi tanda. Konsekuensinya menarik: hanya titik di dekat batas yang menentukan posisi garisnya. Titik yang jauh di dalam wilayahnya sendiri boleh digeser-geser tanpa mengubah hasil. Itu berbeda dari logistic regression, yang setiap titiknya ikut menyumbang ke loss.”

#### Slide 46/88 SVM: margin, C, dan kernel

**Ucapkan:** “Tiga istilah. Margin adalah lebar jalan antara batas keputusan dan support vector terdekat, dan pelatihan memaksimalkan lebar itu. C mengatur seberapa banyak titik boleh masuk ke dalam margin: C kecil memberi margin lebar dengan beberapa pelanggaran, C besar memaksa margin sempit yang rapat mengikuti data train, dan itu arah menuju overfitting. Kernel R B F dipakai kalau kelasnya tidak bisa dipisah garis lurus: data diangkat ke ruang berdimensi lebih tinggi, tempat kedua kelas bisa dipisahkan bidang datar, dan kembali ke dua feature semula batasnya terlihat melengkung.  $\gamma$  pada R B F mengatur seberapa lokal pengaruh satu titik:  $\gamma$  besar berarti tiap titik hanya berpengaruh di sekitarnya sendiri.”

#### Slide 47/88 SVM: kapan pakai dan jebakannya

**Ucapkan:** “Cocok kalau jumlah feature banyak dibanding jumlah baris, kalau batas antar kelasnya cukup tegas, dan kalau kamu bersedia mencari pasangan C dan  $\gamma$  lewat cross-validation, karena tanpa itu hasil SVM sering mengecewakan. Kurang cocok kalau data train-nya ratusan

ribu baris, kalau probabilitas kelas dibutuhkan langsung dari model, atau kalau hasilnya harus dijelaskan sebagai aturan per feature. Jebakannya dua dan keduanya soal praktik: kernel R B F melambat cepat saat data membesar, jadi di nb05 kita pakai subset kalau perlu. Dan SVM mengukur jarak, jadi kalau skala feature berbeda, lakukan scaling lebih dulu.”

#### Slide 48/88 Empat model, satu dataset

**Ucapkan:** ”Data yang sama, empat batas keputusan yang berbeda. Panel decision tree: batasnya bertangga, sejajar sumbu, sesuai penjelasan tadi. KNN dengan K sama dengan lima: batasnya berlekuk mengikuti sebaran data. SVM linear: satu garis lurus. SVM dengan kernel R B F: satu lengkungan mulus. Akurasinya berdekatan, tiga model di nol koma sembilan tujuh dua dan SVM linear di nol koma sembilan empat empat. Yang perlu dibaca dari sini bukan siapa juaranya. Selisih tiga model teratas nol, dan ini satu dataset kecil dengan satu split. Yang benar-benar terlihat adalah bentuk batasnya berbeda karena cara modelnya memutuskan berbeda. Karena itu perbandingan model selalu di satu split dan satu metrik yang sama.”

#### Slide 49/88 Ringkasan Act 4: kapan pakai model yang mana

**Ucapkan:** ”Tabel ini bisa difoto, dan isinya rangkuman empat blok yang baru kita lewati. Baca kolom demi kolom untuk satu model. Logistic regression: batas lurus, perlu scaling, prediksi cepat, mudah dijelaskan. Decision tree: batas sejajar sumbu feature, tidak perlu scaling, mudah dijelaskan. KNN: batasnya mengikuti sebaran data, perlu scaling, dan prediksinya melambat saat data besar. SVM linear dan R B F: keduanya perlu scaling, yang R B F paling lambat dan paling sulit dijelaskan. Kolom scaling berlaku kalau skala feature memang berbeda-beda. Tabel ini membantu memilih kandidat awal. Keputusannya tetap dari hasil di data validation, bukan dari urutan baris di tabel.”

### Act 5 — Bagaimana jika model bekerja sama?

#### Slide 50/88 Act 5: Bagaimana jika model bekerja sama?

**Ucapkan:** ”Sampai sini satu prediksi selalu datang dari satu model. Blok ini menggabungkan beberapa model, dan kita lihat kapan penggabungan itu menolong serta kapan tidak menambah apa-apa.”

#### Slide 51/88 Intuisi: Rata-rata banyak tebakan jatuh lebih dekat

**Ucapkan:** ”Ada percobaan lama di pasar ternak: banyak orang menebak berat seekor sapi. Titik-titik abu-abu ini tebakan mereka, tersebar dari empat ratus sampai tujuh ratus kilogram. Titik merah satu penebak, dan dia melenceng jauh. Garis hijau berat sebenarnya, dan garis putus-putus rata-rata semua tebakan. Lihat betapa dekat keduanya. Yang bekerja di sini bukan

keajaiban, tetapi arah kesalahan: ada yang menebak terlalu tinggi, ada yang terlalu rendah, dan saat dirata-ratakan keduanya saling menghapus. Itu juga syarat yang harus dipenuhi model kita. Gabungan menolong kalau kesalahan tiap model jatuh ke arah yang berbeda. Kalau semuanya melenceng ke arah yang sama, gabungannya ikut melenceng.”

#### **Slide 52/88** Dua cara menggabungkan, dua masalah yang berbeda

**Ucapkan:** ”Dua keluarga besar, dan keduanya menyerang masalah yang berbeda. Bagging melatih banyak model secara terpisah pada data yang sedikit berbeda. Masing-masing tidak stabil, tetapi rata-ratanya jauh lebih tenang, jadi yang diturunkan variance. Boosting melatih model berurutan; tiap model baru mengerjakan kesalahan yang masih tersisa, sehingga pola yang tadinya terlewat mulai tertangkap, dan yang diturunkan bias. Batasnya perlu disebut sekarang supaya tidak kaget nanti. Bagging tidak banyak menolong kalau modelnya memang terlalu sederhana untuk datanya, karena merata-ratakan model yang sama-sama melewatkan pola tetap melewatkan pola. Dan boosting yang dibiarkan menambah model terus akhirnya mengikuti noise di data train, jadi jumlah modelnya perlu dibatasi dari hasil di data validation.”

#### **Slide 53/88** Intuisi: Voting: tiga model, satu keputusan

**Ucapkan:** ”Cara penggabungan yang paling sederhana. Tiga model yang sudah dilatih, masing-masing memberi jawaban untuk satu baris data: logistic regression bilang kelas A, decision tree bilang kelas B, SVM bilang kelas A. Kotak voting hanya menghitung suara, dan keluarannya kelas A dengan dua lawan satu. Yang perlu jelas: voting tidak membuat model baru dan tidak melatih apa pun. Yang dilakukan hanya mengumpulkan keputusan model yang sudah ada lalu mengambil yang terbanyak. Karena itu voting murah dan mudah dicoba di akhir eksperimen, saat kamu sudah punya beberapa model yang hasilnya berdekatan.”

#### **Slide 54/88** Hard voting dan soft voting

**Ucapkan:** ”Dua rasa. Hard voting: setiap model memberi satu suara berupa kelas, dan kelas dengan suara terbanyak yang dipakai; itu contoh di slide tadi. Soft voting: yang dirata-ratakan bukan kelasnya, tetapi probabilitas tiap kelas dari semua model, lalu kelas dengan rata-rata tertinggi yang dipakai. Ini hanya bisa kalau setiap model memang mengeluarkan probabilitas. Soft voting sering sedikit lebih baik karena model yang ragu-ragu, misalnya menjawab nol koma lima satu, tidak berbobot sama dengan model yang yakin di nol koma sembilan sembilan. Selisihnya tetap perlu dicek di data validation. Jebakannya: tiga model yang mirip menghasilkan kesalahan di baris yang sama, jadi voting-nya tidak menambah apa-apa. Yang menolong adalah model yang cara kerjanya berbeda.”

**Slide 55/88**    **Intuisi: Random Forest: banyak pohon dari data yang sedikit berbeda**

**Ucapkan:** "Kalau voting butuh model yang berbeda-beda, dari mana kita dapat banyak model berbeda kalau punyanya cuma satu jenis, decision tree? Jawabannya: beri tiap pohon data yang sedikit berbeda. Caranya bootstrap, dan ini istilah yang perlu tepat. Bootstrap berarti sampel dengan pengembalian: ambil baris acak sebanyak ukuran data semula, dan satu baris boleh terpilih lebih dari sekali. Jadi ada baris yang muncul berulang di satu sampel, dan ada yang tidak terambil sama sekali. Ini bukan membuat data baru, dan tidak ada informasi tambahan yang masuk. Karena datanya tidak persis sama, pohonnya tumbuh berbeda dan salahnya di baris yang berbeda. Lalu semuanya digabung jadi satu jawaban."

**Slide 56/88**    **Random Forest: bootstrap, subset feature, lalu voting**

**Ucapkan:** "Empat langkah. Satu, ambil sampel bootstrap dari data train untuk pohon ini. Dua, tumbuhkan pohonnya seperti biasa, tetapi dengan satu batasan tambahan: di setiap pertanyaan hanya sebagian feature yang dipilih acak yang boleh dipertimbangkan. Batasan kedua ini penting, karena tanpa itu semua pohon akan memilih feature terkuat di simpul atas dan hasilnya jadi mirip-mirip. Tiga, ulangi untuk semua pohon; pohon-pohonnya tidak saling melihat, jadi bisa dilatih paralel. Empat, hasil akhirnya kelas dengan suara terbanyak untuk klasifikasi, rata-rata prediksi untuk regresi. Jebakannya: ratusan pohon di data besar membuat pelatihan dan prediksi terasa lambat, dan pohonnya tidak lagi bisa dibaca satu per satu seperti decision tree tunggal."

**Slide 57/88**    **Intuisi: Boosting: model berikutnya mengerjakan yang masih salah**

**Ucapkan:** "Bandingkan dengan random forest tadi. Di random forest semua pohon bekerja bersamaan dan tidak saling tahu. Di boosting modelnya berurutan. Model satu dilatih dulu dan banyak salahnya. Baris yang masih salah diberi bobot lebih besar, sehingga model dua memusatkan perhatian ke sana. Sisa kesalahan model dua diserahkan ke model tiga, begitu seterusnya. Jadi tiap tambahan model menutup sisa kesalahan model sebelumnya, bukan mengulang pekerjaan yang sama. Satu perbedaan yang perlu disebut sekarang, karena tiga slide berikutnya soal XGBoost: yang saya jelaskan tadi, menaikkan bobot baris yang salah, itu cara AdaBoost. Di gradient boosting, yang dipakai XGBoost, bobot barisnya tidak diubah; model berikutnya justru dilatih untuk mempelajari sisa error, residual, dari model sebelumnya. Satu catatan tentang gambar: label di panel terakhir hanya penanda arah cerita, bukan hasil pengukuran. Seberapa jauh boosting membaik, dan sampai model ke berapa, itu yang kita ukur di nb07 dengan data validation."

**Slide 58/88**    **Bagging dan boosting pada data yang sama**

**Ucapkan:** "Dua panel, data yang sama, dua cara belajar yang berbeda: random forest di kiri, gradient boosting di kanan. Yang menarik justru hasilnya, bentuk batas keputusannya mirip. Jadi cara belajar yang berbeda tidak selalu memberi batas yang berbeda, terutama pada data yang polanya jelas. Sekarang soal angka di bawah tiap panel. Keduanya seratus persen, dan itu dihitung pada data yang sama yang dipakai melatih. Angka seperti itu bukan ukuran kinerja di data

baru, dan hampir selalu bisa dicapai oleh model yang cukup lentur. Kalau kamu melihat seratus persen di laporan seseorang, pertanyaan pertama bukan modelnya apa, tetapi diukur di data mana.”

#### **Slide 59/88** XGBoost: gradient boosting yang siap pakai

**Ucapkan:** ”XGBoost adalah gradient boosting dengan implementasi yang jauh lebih cepat. Pencarian ambang split tidak mencoba semua nilai satu per satu, tetapi dikerjakan dari histogram, dan bebannya bisa dibagi ke banyak core atau ke GPU. Regularisasi sudah ada di dalamnya, jadi ukuran tiap pohon dan besar koreksi tiap langkah bisa ditahan. Satu kenyamanan praktis: nilai kosong tidak perlu diisi lebih dulu, karena tiap split menentukan sendiri ke arah mana baris dengan nilai kosong dikirim. Reputasinya memang kuat di kompetisi data tabular, dan itu wajar disebut. Tapi di data sendiri, hasilnya tetap harus dibandingkan dengan model yang lebih sederhana; kadang selisihnya kecil dan tidak sebanding dengan biaya penyetelannya.”

#### **Slide 60/88** Early stopping: berhenti saat data validation tidak membaik

**Ucapkan:** ”Boosting menambah pohon satu per satu, jadi pertanyaannya berapa banyak. Grafik ini menjawabnya. Kurva hijau error di data train, terus turun selama pohon ditambah, dan itu wajar. Kurva oranye error di data validation: turun sampai satu titik, lalu naik lagi. Garis putus-putus vertikal itu tempat berhentinya. Mekanismenya begini: pelatihan dihentikan kalau tidak ada perbaikan selama sejumlah ronde, dan model dari ronde terbaik yang dipakai, bukan ronde terakhir. Jadi early stopping bukan hanya penghemat waktu, tetapi juga cara memilih jumlah pohon dari data. Yang perlu dijaga: potongan yang dipakai memantau harus validation, bukan test. Kalau test dipakai di sini, angka akhirnya sudah tidak bersih lagi.”

#### **Slide 61/88** Feature importance: kontribusi menurut metode

**Ucapkan:** ”Delapan feature teratas dari XGBoost pada data income. Batang paling atas jauh lebih panjang dari sisanya: kolom status pernikahan, sekitar nol koma tiga delapan; sisanya di bawah nol koma nol enam. Cara membacanya perlu hati-hati. Angka ini menunjukkan seberapa besar feature itu dipakai oleh model ini pada data ini, bukan sebab-akibat, dan bukan pernyataan tentang dunia. Dua hal lagi. Kolom yang saling berkaitan membagi skornya, jadi satu kolom bisa terlihat kecil padahal informasinya sudah masuk lewat kolom lain. Dan skornya berubah kalau modelnya berganti atau cara encoding kolom kategorinya berubah. Jadi pakai daftar ini untuk mengarahkan pemeriksaan berikutnya, bukan sebagai kesimpulan.”

#### **Slide 62/88** Stacking: satu model belajar memakai prediksi model lain

**Ucapkan:** ”Voting menghitung suara dengan aturan tetap. Stacking mengganti aturan tetap itu dengan model. Tiga model dasar di kiri, random forest, XGBoost, dan logistic regression, masing-masing menghasilkan prediksi. Prediksi itu yang jadi masukan meta-model di tengah, dan meta-model belajar sendiri siapa yang lebih bisa dipercaya, bahkan bisa berbeda per wilayah data.

Perhatikan keterangan di bawah kotak: masukannya prediksi, bukan feature asli. Ini berguna saat beberapa model sama baiknya dengan cara yang berbeda dan selisih kecil masih berharga. Satu syarat teknis yang mudah dilanggar: prediksi untuk melatih meta-model harus diambil dari fold yang tidak dipakai melatih model dasarnya. Kalau tidak, meta-model melihat prediksi pada data yang sudah dihafal model dasar.”

#### **Slide 63/88** Tidak ada model terbaik untuk semua data

**Ucapkan:** ”Pertanyaan yang paling sering muncul setelah blok ini: jadi model mana yang terbaik? Jawabannya bergantung pada datanya, dan tiga baris ini pola yang biasa. Data kecil dengan sedikit kolom: model sederhana sering sudah cukup, dan lebih mudah dijelaskan. Data tabular besar dengan kolom campuran angka dan kategori: boosting layak dicoba lebih dulu. Hasil yang harus dijelaskan per feature: model yang bisa dibaca lebih berguna walaupun skornya sedikit di bawah. Baris hijau paling bawah yang paling penting: yang menentukan tetap hasil di data validation, bukan urutan di daftar ini. Dan yang ikut jadi bagian keputusan bukan satu angka metrik saja, tetapi juga waktu latih, waktu prediksi, dan kebutuhan menjelaskan hasil.”

#### **Slide 64/88** Ringkasan Act 5

**Ucapkan:** ”Empat metode, dan tabel ini memasangkan tiap metode dengan masalah yang diselesaikannya. Voting menggabungkan keputusan beberapa model berbeda, menurunkan variance, dan yang dijaga adalah modelnya jangan mirip. Bagging, dalam bentuk random forest, memakai banyak pohon dari sampel bootstrap, juga menurunkan variance, dan yang dijaga jumlah pohon serta waktu latihnya. Boosting, dalam bentuk XGBoost, melatih model berurutan untuk menutup sisa kesalahan, menurunkan bias, dan yang dijaga kapan berhentinya lewat early stopping. Stacking memakai prediksi model dasar sebagai masukan, bisa menyentuh keduanya, dan yang dijaga prediksinya harus dari fold lain. Praktiknya: nb06 memakai voting, bagging, dan boosting pada satu dataset; nb07 masuk ke XGBoost, early stopping, dan feature importance.”

## **Act 6 — Bagaimana menemukan pola tanpa label?**

#### **Slide 65/88** Act 6: Bagaimana menemukan pola tanpa label?

**Ucapkan:** ”Sekarang kolom targetnya kita ambil. Tidak ada jawaban benar yang bisa dihitung, dan itu mengubah cara bekerja sekaligus cara menilai hasilnya.”

#### **Slide 66/88** Ada kolom target, atau tidak ada

**Ucapkan:** ”Perbedaannya bukan soal algoritma, tetapi soal data yang tersedia. Baris pertama: supervised punya kolom target, unsupervised tidak. Baris kedua: yang dicari supervised adalah aturan dari feature ke target; yang dicari unsupervised adalah struktur kelompok di dalam datanya.

Baris ketiga yang paling berdampak ke pekerjaan kita: di supervised, penilaian dilakukan dengan membandingkan prediksi dan nilai sebenarnya. Di unsupervised tidak ada nilai sebenarnya, jadi kita pakai ukuran seperti silhouette ditambah pemeriksaan manual. Konsekuensinya: hasil clustering dinilai dari seberapa berguna kelompoknya untuk keputusan berikutnya, dan itu tidak bisa diputuskan hanya dari satu skor.”

**Slide 67/88**    **Intuisi: K-Means: kelompokkan ke pusat terdekat, lalu geser pusatnya**

**Ucapkan:** ”Gambaranya hasil akhir: titik-titik berwarna menurut kelompoknya, dan tanda silang merah pusat setiap kelompok. Prosesnya begini, dan hanya dua langkah yang diulang. Kelompokkan setiap baris ke pusat yang terdekat. Lalu hitung ulang setiap pusat sebagai rata-rata anggota kelompoknya. Ulangi. Ada satu hal yang sering salah dibayangkan. Titik datanya diam, tidak tertarik ke mana-mana. Yang berubah tiap putaran hanya dua: keanggotaan kelompok dan posisi pusatnya. Kalau setelah satu putaran tidak ada baris yang berpindah kelompok, prosesnya selesai. Tanya ke kelas: kalau semua pusat awal ditaruh berdekatan di satu sudut, menurutmu apa yang terjadi?”

**Slide 68/88**    **K-Means: empat langkah yang diulang**

**Ucapkan:** ”Versi yang bisa ditulis jadi kode. Satu, tentukan K, lalu tempatkan K pusat awal. Dua, kelompokkan setiap baris ke pusat yang paling dekat. Tiga, hitung ulang setiap pusat sebagai rata-rata anggota kelompoknya. Empat, ulangi langkah dua dan tiga sampai keanggotaannya tidak berubah lagi. Perhatikan langkah satu: K harus kita tentukan sebelum jalan, dan algoritma ini tidak akan memberitahu kalau angkanya salah. Yang juga sering dilewati: pusat awal ikut menentukan hasil. Dari pusat awal berbeda, prosesnya bisa berhenti di kelompok yang berbeda. Karena itu `n_init` di scikit-learn menjalankan seluruh proses beberapa kali dengan pusat awal berbeda, lalu mengambil hasil dengan inertia terkecil.”

**Slide 69/88**    **K-Means: kapan pakai dan jebakannya**

**Ucapkan:** ”Cocok kalau ukuran kelompoknya mirip dan bentuknya membulat, kalau semua feature berupa angka dengan skala setara, dan kalau K bisa ditentukan dari kebutuhan atau dicari. Kurang cocok kalau bentuk kelompoknya memanjang atau melengkung, kalau sebagian feature berupa kategori, atau kalau ada outlier yang menarik pusat menjauh; satu pelanggan raksasa bisa menggeser pusat satu kelompok penuh. Jebakannya berasal dari satu sebab yang sama: k-means bekerja dengan jarak dan rata-rata. Karena itu kelompok yang keluar cenderung bulat dan seukuran. Kolom kategori tidak punya arti jarak, jadi tidak bisa dimasukkan apa adanya. Dan kolom belanja yang sebarannya miring perlu diubah dengan `log1p` lebih dulu, seperti yang kita lakukan di nb08.”



#### Slide 70/88    Memilih K: kurva inertia dan titik sikunya

**Ucapkan:** "Kalau K harus kita tentukan, bagaimana memilihnya? Cara pertama: inertia, total jarak kuadrat setiap baris ke pusat kelompoknya. Angkanya di grafik ini turun dari sekitar seribu delapan ratus di K sama dengan dua sampai sekitar seribu di K sama dengan delapan. Dan itu masalahnya: inertia selalu turun saat K bertambah, karena kelompok yang lebih banyak selalu lebih rapat. Kalau yang dicari nilai terkecil, jawabannya selalu satu kelompok per baris. Jadi yang dicari bukan nilai terkecil, melainkan tempat penurunannya mulai melandai, yang biasa disebut titik siku. Pada data Wholesale dengan kolom belanja berskala log, kurvanya turun mulus tanpa siku yang tegas. Jadi cara ini saja belum memutuskan apa pun."

#### Slide 71/88    Memilih K: skor silhouette

**Ucapkan:** "Cara kedua menanyakan hal yang lebih tepat. Silhouette mengukur, untuk setiap baris, seberapa dekat baris itu ke anggota kelompoknya sendiri dibanding ke kelompok tetangga terdekat. Rentangnya minus satu sampai satu; makin tinggi makin jelas pemisahannya. Di data ini skornya paling tinggi di K sama dengan dua, sekitar nol koma dua sembilan, lalu turun untuk K yang lebih besar. Yang perlu diingat, angka ini satu bahan pertimbangan, bukan perintah. Kalau kebutuhan bisnis meminta segmen yang lebih halus, K sama dengan tiga masih wajar, skornya sekitar nol koma dua enam; tetapi di K sama dengan empat skornya turun jelas ke sekitar nol koma satu sembilan, dan penurunan itu bagian dari keputusan yang kita ambil sadar. Yang tidak boleh: memilih K dari skor lalu menceritakannya sebagai jumlah kelompok yang sebenarnya ada."

**Baca rumus:**

- Rumusnya tidak ditampilkan di slide. Kalau ada yang bertanya:  $s = \frac{b - a}{\max(a, b)}$  dibaca "s sama dengan b kurang a, per, maksimum dari a dan b".
- Sebut artinya: "a rata-rata jarak ke anggota kelompoknya sendiri, b rata-rata jarak ke anggota kelompok tetangga terdekat". Skor keseluruhan adalah rata-rata s untuk semua baris.

#### Slide 72/88    Hierarchical clustering: gabungkan yang terdekat, lalu potong

**Ucapkan:** "Metode kedua bekerja dari bawah. Mulai dari tiap baris sebagai kelompok sendiri: A sampai F di gambar ini. Gabungkan dua kelompok yang terdekat, di sini A dengan B, C dengan D, E dengan F. Lalu gabungan-gabungan itu digabung lagi, sampai tinggal satu kelompok besar. Gambar seperti ini disebut dendrogram, dan tinggi tiap sambungan menunjukkan seberapa jauh dua kelompok itu saat digabung; sambungan yang tinggi berarti dua kelompok yang sebenarnya tidak mirip. Jumlah kelompoknya tidak ditentukan di awal, tetapi didapat dengan memotong dendrogram di satu ketinggian. Garis oranye itu memotong dua cabang, jadi hasilnya dua kelompok. Satu pilihan yang berpengaruh besar: **linkage** menentukan arti terdekat antar kelompok."

### Slide 73/88 DBSCAN: kelompok dari kepadatan, sisanya outlier

**Ucapkan:** "Metode ketiga berangkat dari kepadatan, bukan dari jarak ke pusat. Aturannya: baris yang punya paling sedikit `min_samples` titik di dalam radius `eps`, lingkaran oranye itu, menjadi inti kelompok, dan tetangganya ikut bergabung. Titik yang di tengah lingkaran itu ikut dihitung sebagai salah satunya, jadi `min_samples` sama dengan lima berarti titik itu sendiri tambah empat tetangga. Kelompok tumbuh dari inti ke inti. Dua sifat yang membedakannya dari k-means. Pertama, baris yang tidak masuk kelompok mana pun ditandai sebagai outlier, tanda silang merah itu, bukan dipaksa masuk ke kelompok terdekat. Kedua, jumlah kelompoknya tidak ditentukan di awal dan bentuknya bebas, jadi kelompok memanjang pun bisa ditemukan. Harganya: dua parameter yang harus dipilih. `eps` diambil dari k-distance plot, yaitu urutkan jarak ke tetangga ke-k lalu ambil nilai di tempat kurvanya mulai menanjak."

### Slide 74/88 Clustering di data nyata: yang mudah terlewat

**Ucapkan:** "Dua hal yang mudah terlewat, dan keduanya ada di data Wholesale yang dipakai nb08. Pertama, data itu punya kolom `Channel` dan `Region` yang isinya angka kode, misalnya satu dan dua. Kalau keduanya ikut sebagai feature, k-means menghitung selisih angka kode itu sebagai jarak, padahal selisih antara kode satu dan kode dua tidak berarti apa-apa. Hasil clusternya jadi hampir hanya meniru `Channel`. Kedua, enam kolom belanja sebarannya miring: sedikit pelanggan besar menarik pusat kelompok ke arah mereka. Yang kita kerjakan di nb08: pakai enam kolom belanja saja, ubah dengan `log1p`, jalankan k-means, lalu bandingkan hasilnya dengan `Channel` lewat cross-tab sebagai pemeriksaan, bukan sebagai target."

### Slide 75/88 Ringkasan Act 6

**Ucapkan:** "Tiga metode, dan bedanya paling cepat dilihat dari kolom-kolom ini. K-Means perlu K di awal, kelompoknya cenderung bulat, outlier tidak ditandai, dan yang kita pilih K serta `n_init`. Hierarchical tidak perlu K di awal, bentuknya lebih fleksibel, outlier juga tidak ditandai, dan yang kita pilih `linkage` serta tinggi potongnya. DBSCAN tidak perlu K, bentuk kelompoknya bebas, outlier ditandai, dan yang kita pilih `eps` serta `min_samples`. Ketiganya dipakai di nb08 pada data yang sama supaya bisa dibandingkan. Yang menentukan pilihan bukan skornya saja, tetapi juga apakah kelompoknya bisa dijelaskan ke orang yang akan memakainya."

## Act 7 — Kapan GPU membantu?

### Slide 76/88 Act 7: Kapan GPU membantu?

**Ucapkan:** "Blok terakhir sebelum penutup, dan isinya soal kecepatan. Pertanyaannya bukan apakah GPU lebih cepat, tetapi untuk pekerjaan seperti apa, dan bagaimana mengukur selisihnya supaya angkanya bisa dipercaya."

#### Slide 77/88    Intuisi: CPU dan GPU: satu koki ahli atau seribu koki sederhana

**Ucapkan:** "CPU punya empat sampai enam belas core, masing-masing kuat dan bisa mengerjakan pekerjaan rumit yang alurnya berliku. Anggap satu koki ahli: bisa masak menu apa saja, satu per satu. GPU punya ribuan core yang jauh lebih sederhana. Anggap seribu koki yang cuma bisa satu tugas, misalnya memotong bawang, tetapi mengerjakannya serentak. Nah, melatih model itu pekerjaan yang mana? Ternyata lebih dekat ke tumpukan bawang: mengulang operasi kecil yang sama pada jutaan baris. Pekerjaan seperti itu bisa dibagi ke banyak core sekaligus. Yang tidak cocok adalah pekerjaan yang tiap langkahnya bergantung pada hasil langkah sebelumnya, karena tidak ada yang bisa dikerjakan serentak."

#### Slide 78/88    Pekerjaan seperti apa yang cocok di GPU

**Ucapkan:** "Tiga syarat di kolom kiri, dan ketiganya perlu terpenuhi bersama. Operasinya sama untuk semua baris, misalnya perkalian matriks atau perhitungan jarak seperti di KNN dan k-means. Barisnya banyak, sehingga semua core punya bagian. Dan datanya sudah berada di memori GPU. Kolom kanan kebalikannya: data kecil beberapa ribu baris, alur yang bercabang berbeda-beda per baris, atau data yang harus bolak-balik dipindah antara CPU dan GPU. Untuk hari ini, yang dijalankan di GPU pada nb10 ada empat: Random Forest dan KNN dari cuML, K-Means, serta XGBoost dengan `device="cuda"`. Keempatnya masuk kategori kolom kiri, dan itu bukan kebetulan."

#### Slide 79/88    cuML: yang berubah hanya baris import-nya

**Ucapkan:** "Ini slide kode pertama dari dua di deck ini, dan sengaja hanya dua, karena yang mau saya tunjukkan justru betapa sedikit yang berubah. Dua baris import di atas: yang pertama dari `sklearn.ensemble`, yang kedua dari `cuml.ensemble`. Nama kelasnya sama, `RandomForestClassifier`. Tiga baris di bawahnya, membuat model, `fit()`, lalu `predict()`, tidak berubah sama sekali. Jadi kode setelah import bisa dibiarkan apa adanya. Dua hal yang perlu disiapkan sebelum kaget di notebook. Tidak semua argumen scikit-learn tersedia di cuML, jadi sebagian pengaturan perlu dicek di dokumentasinya. Dan hasilnya bisa berbeda sedikit dari versi CPU karena urutan perhitungannya berbeda."

#### Slide 80/88    XGBoost di GPU: satu argumen

**Ucapkan:** "Untuk XGBoost lebih sederhana lagi: satu argumen. `device` sama dengan `cpu` atau `cuda`, dan sisa kodenya sama, `fit()`, `predict()`, dan metriknya tidak berubah. Dua catatan praktis. Kalau GPU tidak tersedia, `device="cuda"` akan gagal atau jatuh kembali ke CPU tanpa kamu sadari, jadi di nb10 kita cek dulu ketersediaan GPU dan mencetak hasilnya, supaya tidak mengira sudah mengukur GPU padahal masih CPU. Kedua, hasil prediksinya bisa berbeda sedikit dari versi CPU, karena urutan penjumlahan bilangan pecahan berbeda. Selisih di angka desimal ke belakang itu normal dan bukan tanda ada yang salah."

## Slide 81/88 Kapan GPU tidak menang

**Ucapkan:** "Bagian ini yang paling sering hilang dari materi tentang GPU, padahal paling berguna. Data harus dipindah ke memori GPU lebih dulu, dan pemindahan itu punya biaya; kalau pekerjaannya singkat, waktu pindahnya lebih besar daripada waktu yang dihemat. Panggilan pertama selalu terasa lambat karena kernel dimuat dan memori dialokasikan; itu warm-up, dan bukan bagian yang diukur. Pada data kecil, satu core CPU yang kuat sering menyelesaikan lebih cepat daripada ribuan core yang menunggu data. Dan pekerjaan yang alurnya bercabang berbeda per baris tidak bisa dibagi rata. Jadi urutan yang benar saat mengukur: jalankan sekali untuk warm-up, baru catat waktunya. Kalau hasilnya CPU lebih cepat, itu temuan yang sah, bukan kesalahan."

## Slide 82/88 Benchmark: syarat pengukurannya disebut

**Ucapkan:** "Ini hasil pengukuran sungguhan dari nb10 di Colab dengan Tesla T4, tanggal tujuh September. Sebelum membaca angkanya, baca dulu syaratnya di kanan: tugasnya Random Forest dua ratus pohon, XGBoost, KNN, K-Means, dan PCA; datanya Covertypes lima ratus delapan puluh satu ribu baris, khusus KNN subset seratus ribu baris; yang diukur waktu `fit()` saja, setelah warm-up; perangkatnya satu T4 dibandingkan CPU pada runtime yang sama. Sekarang angkanya. Random Forest: seratus enam puluh dua detik di CPU, sepuluh detik di GPU, enam belas kali lebih cepat. XGBoost sembilan kali. KNN paling dramatis, dua puluh dua kali, karena menghitung jarak ke banyak titik memang pekerjaan paralel. K-Means tiga setengah kali. Lihat baris paling atas: PCA justru kalah, nol koma dua kali. Operasinya selesai dalam seperempat detik di CPU, jadi biaya memindahkan data ke GPU lebih besar daripada kerjanya. Itu bukan GPU yang jelek, itu pekerjaan yang terlalu kecil. Satu catatan lagi soal akurasi: XGBoost dan KNN skornya identik di CPU dan GPU, tapi Random Forest cuML dapat nol koma tujuh lima sembilan, sedangkan sklearn nol koma tujuh delapan tiga. Itu karena cuML memakai implementasi yang berbeda, dengan pembagian histogram, bukan sekadar versi cepat dari sklearn. Kalau kamu pindah ke GPU, cek ulang skornya, jangan dianggap otomatis sama. Sumbu waktunya skala log supaya PCA yang seperempat detik dan Random Forest yang seratus enam puluh detik bisa muat di satu gambar."

## Slide 83/88 Ekosistem NVIDIA di sekitar pekerjaan ini

**Ucapkan:** "Enam nama yang akan sering kamu dengar, supaya tidak bingung mana yang untuk apa. cuML: algoritma ML klasik di GPU dengan API mirip scikit-learn, dan itu yang kita pakai. cuDF: operasi dataframe di GPU, mengikuti gaya pandas, jadi pasangannya cuML. XGBoost GPU sudah kita lihat, cukup lewat argumen `device`. Tiga sisanya di luar hari ini: TensorRT mengoptimalkan model deep learning untuk inferensi, NeMo untuk melatih dan menyetel model bahasa, dan Triton untuk melayani model terlatih di produksi. Hari ini yang benar-benar dipakai hanya cuML dan XGBoost GPU. Tiga nama terakhir dibahas di Modul 6, jadi tidak perlu dihafal sekarang."

## Slide 84/88 Ringkasan Act 7

**Ucapkan:** "Empat hal dari blok ini. GPU membantu kalau pekerjaannya besar, seragam, dan bisa dikerjakan serentak pada banyak baris. Perubahan kodenya kecil: satu baris import untuk cuML, satu argumen `device` untuk XGBoost. Ada keadaan ketika CPU lebih cepat, dan itu bukan kesalahan pengukuran: data kecil, alur bercabang, atau data yang bolak-balik dipindah. Dan yang terakhir ini kebiasaan, bukan pengetahuan: setiap klaim kecepatan menyebut tugasnya, ukuran datanya, perangkatnya, dan bagian mana yang diukur. Kalau salah satu tidak disebut, angkanya belum bisa dibandingkan dengan apa pun."

## Penutup

### Slide 85/88 Sepuluh notebook hari ini

**Ucapkan:** "Sekarang seluruh daftarnya, dan kolom jenisnya yang perlu diperhatikan. Notebook satu regresi. Notebook dua sampai lima klasifikasi: logistic regression, decision tree, KNN, SVM. Notebook enam dan tujuh ensemble. Notebook delapan clustering, dan itu satu-satunya yang unsupervised. Notebook sembilan time series. Notebook sepuluh bukan model baru, tetapi akselerasi: model yang sama dijalankan di GPU. Delapan dari sepuluh baris ini bertanda supervised, jadi alur kerja Act 2 tadi berlaku di hampir semuanya. Setiap notebook memuat datanya sendiri dari URL dan bisa dijalankan berdiri sendiri, jadi kalau satu notebook macet, notebook berikutnya tetap bisa dibuka."

### Slide 86/88 Apa selanjutnya

**Ucapkan:** "Modul satu selesai hari ini, dan besok kita masuk Modul dua, Deep Learning Fundamentals. Yang menyenangkan: istilah hari ini dipakai lagi hampir seluruhnya. Split data, loss, overfitting, cross-validation, dan cara membandingkan model, semuanya berlaku sama. Yang berganti jenis modelnya: bukan lagi garis, pohon, atau jarak, tetapi jaringan berlapis dengan jauh lebih banyak parameter. Lalu Modul tiga N L P, Modul empat L L M, Modul lima R A G, dan Modul enam ekosistem NVIDIA. Jadi kalau hari ini terasa paling banyak istilah barunya, itu memang wajar: fondasinya dipasang sekali dan dipakai lima modul berikutnya."

### Slide 87/88 Latihan mandiri sebelum hari 2

**Ucapkan:** "Sebelum ditutup, satu permintaan. Setiap notebook ditutup satu blok Latihan berisi dua atau tiga tugas, dengan sel kosong bertanda `# TODO` dan satu petunjuk. Kerjakan minimal satu latihan per notebook sebelum besok. Kunci jawabannya tidak disertakan, dan itu memang sengaja: cara memeriksanya adalah menjalankan kodenya, melihat angka yang keluar, lalu membandingkannya dengan baseline di notebook itu. Kalau angkanya di bawah baseline, ada yang perlu diperiksa. Kalau tersangkut pada satu model, buka kembali frame kapan pakai dan jebakannya untuk model itu; hampir semua kemacetan yang saya pernah lihat sudah tertulis di

situ. Untuk nb10 runtime Colab-nya diubah ke GPU; sembilan lainnya cukup CPU.”

**Slide 88/88**   **Terima kasih**

**Ucapkan:** ”Terima kasih sudah bertahan satu hari penuh. Kalau hari ini hanya satu hal yang terbawa, saya berharap yang terbawa adalah alur kerjanya: split, baseline, bandingkan, uji sekali di akhir. Sampai besok di Modul 2, Deep Learning Fundamentals.”